

Document number R3880R0
Date 2025-10-10
Audience LEWG
Reply-to Hewill Kang <hewillk@gmail.com>

Make subspan aware of compile-time constants

- - Abstract
 - Revision history
 - Discussion
 - Proposed change
 - References

Abstract

This paper proposes new overloads of `span::subspan`, `first`, and `last` that accept compile-time constant arguments such as `std::cw`. These overloads allow `span` to recognize compile-time offsets and extents, returning a `span` with a static extent instead of `dynamic_extent`:

```
int arr[42] = {};  
std::span sp(arr);  
auto first5 = sp.first (std::cw<5>);  
auto last5 = sp.last (std::cw<6>);  
auto sub = sp.subspan(std::cw<2>, std::cw<9>);
```

`// span<int, 42>`
`// span<int, 5>`
`// span<int, 6>`
`// span<int, 9>`

This unifies runtime and compile-time interfaces, improves `constexpr` usability, and aligns `span` with `std::mdspan` ecosystems.

Revision history

R0

Initial revision.

Discussion

The primary motivation for these overloads is consistency and expressiveness. In C++26, `mdspan` introduced `submdspan`, which allows slicing using compile-time constants directly as function parameters such as `std::strided_slice`. By contrast, `std::span` currently requires non-type template arguments or runtime integers, leading to inconsistency between the two interfaces.

By introducing overloads that take *integral-constant-like* parameters, `span::first`, `last`, and `subspan` become symmetric with `submdspan`. Users can express static slicing using the same syntax and semantics across both types, improving uniformity in multidimensional and one-dimensional views.

Additionally, this change enables the compiler to deduce the resulting extent at compile time, preserving static extent information and eliminating unnecessary runtime checks. This makes code more type-safe, efficient, and self-documenting—especially when working with fixed-size `spans` or compile-time indexed slicing.

Proposed change

This wording is relative to [N5014](#).

1. Edit 23.7.2.2.1 [\[span.overview\]](#) as indicated:

```
namespace std {  
    template<class ElementType, size_t Extent = dynamic_extent>  
    class span {  
    public:  
        [...]  
        // [span.sub], subviews
```

```

template<size_t Count>
constexpr span<element_type, Count> first() const;
template<size_t Count>
constexpr span<element_type, Count> last() const;
template<size_t Offset, size_t Count = dynamic_extent>
constexpr span<element_type, see below> subspan() const;

template<integral-constant-like Count>
constexpr span<element_type, size_t(Count::value)> first(Count) const;
template<integral-constant-like Count>
constexpr span<element_type, size_t(Count::value)> last(Count) const;
template<integral-constant-like Offset,
integral-constant-like Count = integral_constant<size_t, dynamic_extent>>
constexpr span<element_type, see below> subspan(Offset, Count = {}) const;

constexpr span<element_type, dynamic_extent> first(size_type count) const;
constexpr span<element_type, dynamic_extent> last(size_type count) const;
constexpr span<element_type, dynamic_extent> subspan(
    size_type offset, size_type count = dynamic_extent) const;
    [...]
};
    [...]
}

```

2. Edit 23.7.2.2.4 [\[span.sub\]](#) as indicated:

-10- *Remarks:* The second template argument of the returned `span` type is:

```

Count != dynamic_extent ? Count
    : (Extent != dynamic_extent ? Extent - Offset
    : dynamic_extent)

```

[Drafting note: List initialization are used to ensure the conversion is non-narrowing. - end drafting note]

```

template<integral-constant-like Count>
constexpr span<element_type, size_t(Count::value)> first(Count) const;

```

-?- *Effects:* Equivalent to: `return first<{Count::value}>();`

```

template<integral-constant-like Count>
constexpr span<element_type, size_t(Count::value)> last(Count) const;

```

-?- *Effects:* Equivalent to: `return last<{Count::value}>();`

```

template<integral-constant-like Offset,
integral-constant-like Count = integral_constant<size_t, dynamic_extent>>
constexpr span<element_type, see below> subspan(Offset, Count = {}) const;

```

-?- *Effects:* Equivalent to: `return subspan<{Offset::value}, {Count::value}>();`

-?- *Remarks:* The second template argument of the returned `span` type is:

```

size_t(Count::value) != dynamic_extent ? size_t(Count::value)
    : (Extent != dynamic_extent ? Extent - size_t(Offset::value)
    : dynamic_extent)

```

References

[P2630R4]

Nevin Liber. `submdspan`. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2630r4.html>

[P2781R9]

Zach Laine. `std::constant_wrapper`. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2781r9.html>